Name: Vineet Gajwani
Roll No: 28
Batch: T12, Branch: AI&DS

Experiment No. 1

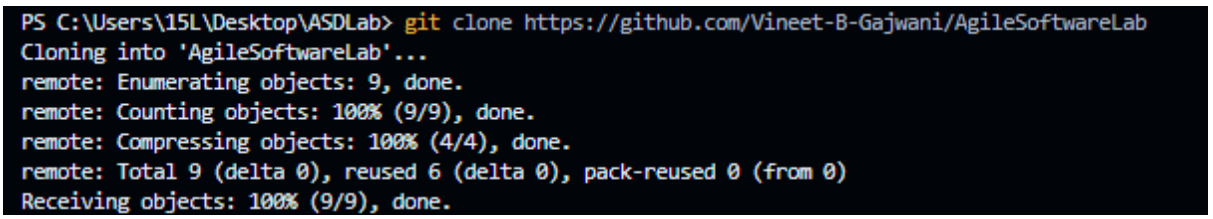
Aim:- To perform version control operations in Git.

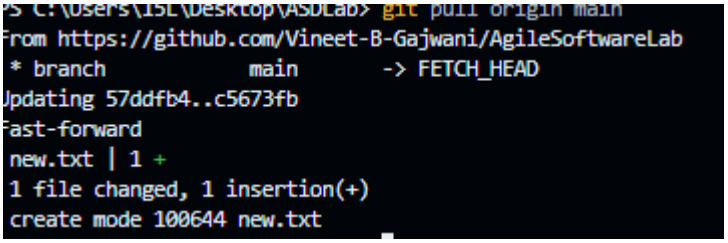
Lab Outcome:-(Lab Outcome 2): Use Git effectively for version control, source code management, and collaborative team development.

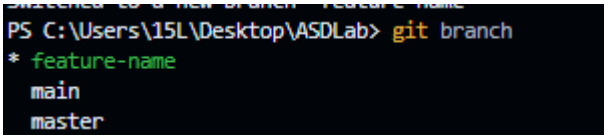
Theory:-
Introduction to Distributed Version Control Systems (DVCS)
Git is a mature, actively maintained open-source distributed version control system. Unlike older centralized systems (like SVN), where the complete history is available only on a single server, every developer's working copy in Git contains a full clone of the repository, including its complete history and track records. This architecture ensures high availability, speed, and seamless offline operations.
Core Workflows and Command Mechanisms
Based on the practical implementation, the following foundational concepts and underlying mechanisms govern Git's operational workflow:
Repository Synchronization and Remote Tracking:
git clone: Used to target an external host and download a mirror of the remote repository architecture locally.
git fetch: Queries the remote tracking branches to pull metadata updates without altering the state of the active local files.
git pull: A compound operation that executes a fetch followed immediately by a merge sequence (git merge) to integrate remote changes into the working branch.
git push: Publishes local commit logs up to the designated upstream tracking branch on a host provider (e.g., GitHub).
Branching Strategies and Environment Switching:
Git implements lightweight branching via pointer references pointing to specific commit hashes. Commands like git switch -c or git checkout -b allow immediate context switching and isolated stream creation (feature-name) without impacting stable deployment pipelines (like main or master).
Inspection, Blame, and Diagnostics:
Staging Area Inspection: Internal database tracking commands such as git ls-files --stage query the index file to inspect file path structures, permissions, and active SHA-1 object keys.
git status: Maps differences between the working directory, index file, and current HEAD state.
git diff: Generates line-by-line granular delta views of tracking modifications.
git blame: Provides an audit trail for specific lines within a file, outputting the author, execution timestamp, and respective commit hash.
Temporary Stashing and File Erasure:
git stash: Temporarily shelving modified, tracked assets to clear the working index for emergency patches.

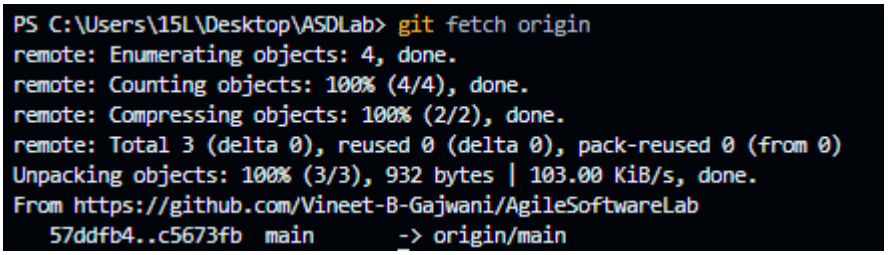
git rm: Schedules a file removal action from both the physical working environment and the immediate index tracking system.

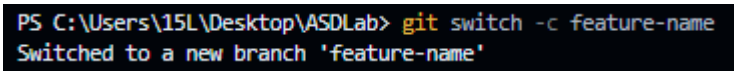
Output Images:

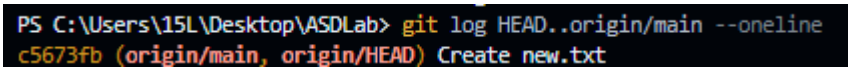


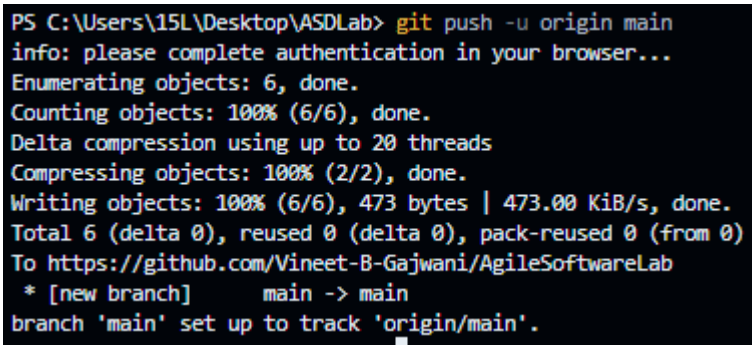


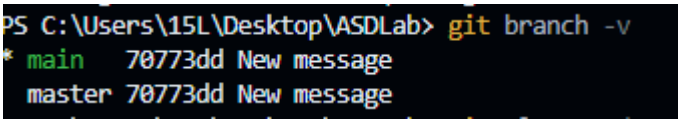












```
PS C:\Users\15L\Desktop\ASDLab> git commit -v -m "Commit with verbose output"
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   sample.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        AgileSoftwareLab/
```

```
PS C:\Users\15L\Desktop\ASDLab> git ls-files --stage
100644 95d09f2b10159347eece71399a7e2e907ea3df4f 0       sample.txt
```

```
PS C:\Users\15L\Desktop\ASDLab> git status --short
 M sample.txt
?? AgileSoftwareLab/
```

```
PS C:\Users\15L\Desktop\ASDLab> git blame sample.txt
^70773dd (Vineet Gajwani 2026-07-14 05:14:03 +0530 1) hello world
```

```
PS C:\Users\15L\Desktop\ASDLab> git diff sample.txt
diff --git a/sample.txt b/sample.txt
index 95d09f2..1f7a84d 100644
--- a/sample.txt
+++ b/sample.txt
@@ -1 +1 @@
-hello world
\ No newline at end of file
+hello world..
\ No newline at end of file
```

```
PS C:\Users\15L\Desktop\ASDLab> git var -l
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=openssl
http.sslcainfo=C:/Program Files/Git/mingw64/etc/ssl/certs/ca-bundle.crt
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=Vineet Gajwani
user.email=your.email@example.com
core.repositoryformatversion=0
core.filemode=false
core.bare=false
core.logallrefupdates=true
core.symlinks=false
core.ignorecase=true
user.name=Vineet Gajwani
remote.origin.url=https://github.com/Vineet-B-Gajwani/AgileSoftwareLab
remote.origin.fetch=+refs/heads/*:refs/remotes/origin/*
GIT_COMMITTER_IDENT=Vineet Gajwani <your.email@example.com> 1783987391 +0530
GIT_AUTHOR_IDENT=Vineet Gajwani <your.email@example.com> 1783987391 +0530
GIT_EDITOR=vi
GIT_SEQUENCE_EDITOR=vi
GIT_PAGER=less
GIT_DEFAULT_BRANCH=master
GIT_SHELL_PATH=C:/Program Files/Git/usr/bin/sh.exe
GIT_ATTR_SYSTEM=C:/Program Files/Git/etc/gitattributes
GIT_ATTR_GLOBAL=C:/Users/15L/.config/git/attributes
GIT_CONFIG_SYSTEM=C:/Program Files/Git/etc/gitconfig
GIT_CONFIG_GLOBAL=C:/Users/15L/.config/git/config
GIT_CONFIG_GLOBAL=C:/Users/15L/.gitconfig
PS C:\Users\15L\Desktop\ASDLab> git help config
PS C:\Users\15L\Desktop\ASDLab> git blame sample.txt
^70773dd (Vineet Gajwani 2026-07-14 05:14:03 +0530 1) hello world
PS C:\Users\15L\Desktop\ASDLab> git diff sample.txt
PS C:\Users\15L\Desktop\ASDLab> git diff sample.txt
diff --git a/sample.txt b/sample.txt
index 95d09f2..1f7a84d 100644
--- a/sample.txt
+++ b/sample.txt
```

```
PS C:\Users\15L\Desktop\ASDLab> git rev-parse --abbrev-ref HEAD
main
```

```
PS C:\Users\15L\Desktop\ASDLab> git count-objects -v
count: 5
size: 0
in-pack: 0
packs: 0
size-pack: 0
prune-packable: 0
garbage: 0
size-garbage: 0
```

```
PS C:\Users\15L\Desktop\ASDLab> git count objects -v
git: 'count' is not a git command. See 'git --help'.

The most similar commands are
        column
        commit
```

```
PS C:\Users\15L\Desktop\ASDLab> git config --list --local
core.repositoryformatversion=0
core.filemode=false
core.bare=false
core.logallrefupdates=true
core.symlinks=false
core.ignorecase=true
user.name=Vineet Gajwani
remote.origin.url=https://github.com/Vineet-B-Gajwani/AgileSoftwareLab
remote.origin.fetch=+refs/heads/*:refs/remotes/origin/*
PS C:\Users\15L\Desktop\ASDLab> git diff --stat
PS C:\Users\15L\Desktop\ASDLab> git reflog
70773dd (HEAD -> main, master) HEAD@{0}: Branch: renamed refs/heads/feature-branch to refs/heads/main
70773dd (HEAD -> main, master) HEAD@{2}: checkout: moving from master to feature-branch
70773dd (HEAD -> main, master) HEAD@{3}: commit (amend): New message
fdf2fde HEAD@{4}: commit (amend): New message
4ac60fd HEAD@{5}: commit (initial): Add empty sample file
```

```
PS C:\Users\15L\Desktop\ASDLab> git log --graph --oneline --decorate
* 70773dd (HEAD -> main, master) New message
```

```
PS C:\Users\15L\Desktop\ASDLab> git add sample.txt
fatal: pathspec 'sample.txt' did not match any files
```

```
PS C:\Users\15L\Desktop\ASDLab> git ls-files
new.txt
sample.txt
PS C:\Users\15L\Desktop\ASDLab>
```

```
PS C:\Users\15L\Desktop\ASDLab> git stash
Saved working directory and index state WIP on feature-name: c5673fb Create new.txt
```

```
PS C:\Users\15L\Desktop\ASDLab> git rm new.txt
rm 'new.txt'
```

```
Vineet Gajwani@LAPTOP-HMASL84U MINGW64 ~/OneDrive/Desktop/New folder/AgileSoftwareLab (main)
$ git merge feature/js-boilerplate
Updating c5673fb..1b095bf
Fast-forward
 index.js | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 index.js
```

```
Vineet Gajwani@LAPTOP-HMASL84U MINGW64 ~/OneDrive/Desktop/New folder/AgileSoftwareLab (main)
$ git log -p
commit 1b095bf3aedef1a72fc0a73e0ddf54dde3d16347 (HEAD -> main, origin/feature/js-boilerplate, feature/js-boilerplate)
Author: Vineet-B-Gajwani <vineetgajwani14@gmail.com>
Date:   Thu Jul 16 19:46:14 2026 +0530

    Index.js

diff --git a/index.js b/index.js
new file mode 100644
index 0000000..cedaca3
--- /dev/null
+++ b/index.js
@@ -0,0 +1 @@
+console.log('Hello, World');

commit c5673fb71a89cd7db7a86ba2298ead4848e9cef6 (origin/main, origin/HEAD)
Author: Vineet-B-Gajwani <vineetgajwani14@gmail.com>
Date:   Tue Jul 14 11:26:51 2026 +0530

    Create new.txt

diff --git a/new.txt b/new.txt
new file mode 100644
index 0000000..e16ebcc
--- /dev/null
+++ b/new.txt
```

```
Vineet Gajwani@LAPTOP-HMASL84U MINGW64 ~/OneDrive/Desktop/New folder/AgileSoftwareLab (main)
$ git reset --hard HEAD
HEAD is now at 1b095bf Index.js
```

Conclusion:-

Through this experiment, I gained practical experience with Git version control by learning how to inspect repository information, analyze commit history, manage branches, compare file changes, use the staging area effectively, and temporarily save work using Git stash. This enhanced my understanding of Git's workflow and its role in efficient project collaboration and version management.